

T.C. Fırat Üniversitesi Mühendislik Fakültesi Yazılım Mühendisliği Bölümü
Veritabanı Yönetim Sistemleri Dersi Proje Raporu

Grup Numarası : 9

Takım Lideri ve Grup Üyelerinin Okul Numaraları, Ad ve Soyadları

- 240290020 - Uğur Erdoğan
- 240290090 - Hilal Çoğul
- 250290096 - Ecenur Eke

Projenin Adı : Otel Otomasyonu

1. PROJE ÖZELLİKLERİ

1.1 Projenin Amacı

Bu projenin amacı; modern bir otel işletmesinin temel operasyonları (rezervasyon ekleme, boş oda kontrolü, otel doluluk kontrolü, müşterilerin ek hizmetlerden yararlanması ve yararlanılan hizmetlerin faturaya yansıtılması, fiyat güncellemelerinin yapılması) ilişkisel bir veritabanı mimarisi üzerinde güvenli bir şekilde optimize etmeyi, veri bütünlüğünü tetikleyiciler (triggers) ve saklı yordamlar (stored procedures) ile veritabanı seviyesinde koruyan bir otel otomasyon sistemi geliştirmektir. Proje, resepsiyonist ve admin rolleri arasında eş zamanlı, sıfır veri kaybı ve minimum gecikmeyle operasyonların yürütülmesini hedefler.

1.2. Veritabanı Bilgileri

Veritabanı Adı: otel_otomasyonu

Veritabanı Yönetim Sistemi: MySQL (İlişkisel veri modeli, View ve Stored Procedure destekleri için kullanılmıştır.)

Veritabanı Sürücüsü: mysql-connector-python (Python ile MySQL arasındaki veri iletişimini sağlamak için entegre edilmiştir.)

Tablo Yapısı ve İlişkisel Şema

1. Oda_Turleri: Odaların kategorilerini (Standart, Süit vb.) ve taban fiyatlarını tutar.
2. Musteriler: Otelde konaklayacak kişilerin temel iletişim ve kimlik bilgilerini barındırır.
3. Personeller: Sistemi (arayüzü) kullanacak otel çalışanlarının ve adminlerin giriş bilgilerini tutar.

4. Hizmetler: Otelin sunduğu ekstra ücretli hizmetlerin (SPA, Minibar vb.) tanımlandığı tablodur.
5. Odalar: Oteldeki fiziksel odaların numaralarını ve anlık durumlarını (Boş/Dolu) takip eder.
6. Rezervasyonlar: Sistemin ana merkezidir. Hangi müşterinin, hangi odada, hangi tarihlerde kalacağını birleştirir.
7. Rezervasyon_Hizmetleri: Bir rezervasyon süresince müşterinin kullandığı ekstra hizmetleri ve adetlerini kaydeder.
8. Faturalar: Rezervasyon bittiğinde kesilen toplam tutar ve ödeme yöntemini tutar.
9. Islem_Kayitlari: Sistemde hangi personelin ne zaman hangi işlemi yaptığını loglayan güvenlik tablosudur.

1.3. Backend için Kullanılan Programlama Dili, Web Framework Bilgileri

Programlama Dili: Python 3

Web Çerçevesi (Framework): FastAPI (Yüksek performanslı, asenkron ve RESTful API geliştirmeye uygun modern mimarisi nedeniyle tercih edilmiştir.)

Sunucu (ASGI Server): Uvicorn (FastAPI uygulamasını ayağa kaldırmak ve asenkron istekleri yönetmek için kullanılmıştır.)

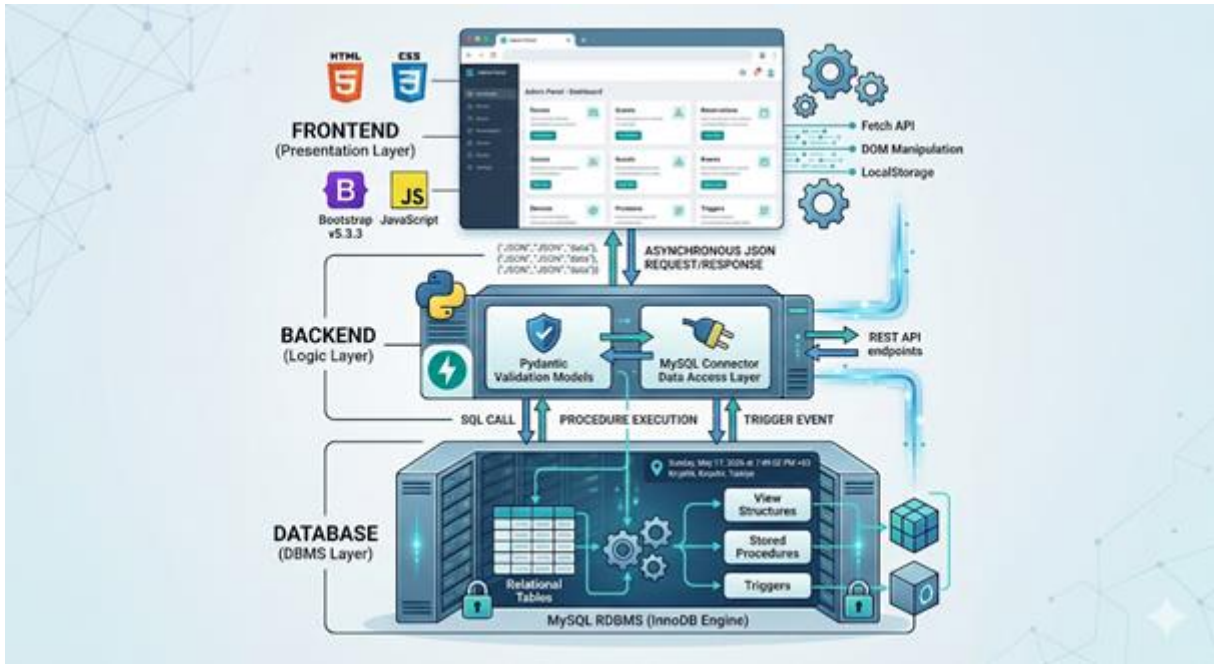
1.4. Frontend için kullanılan araçlar

İşaretleme ve Şekillendirme: HTML5 & CSS3 (Modern, anlamsal ve erişilebilir arayüz yapısı için.)

Programlama Dili: JavaScript (ES6+) (Dinamik sayfa etkileşimleri, asenkron veri akışı ve tarayıcı tabanlı iş mantığı için.)

CSS Çerçevesi (Framework): Bootstrap 5.3 (Grid sistemi, mobil uyumluluk (responsive) ve hazır UI bileşenleri (Modallar, tablolar) için kullanılmıştır.)

1.5. Proje Mimarisinin Gösterimi



Şekil 1: Otel Otomasyon Sistemi Üç Katmanlı (Full-Stack) Mimari ve Veri Akış Şeması

Sistemimiz, modern yazılım mühendisliği standartlarına uygun olarak Üç Katmanlı Mimari (3-Tier Architecture) prensibiyle tasarlanmıştır. Bu yapı; kullanıcı arayüzünü sunan Sunum Katmanı (Frontend), iş kurallarını ve veri doğrulamasını yöneten Mantık Katmanı (Backend) ve verilerin güvenli bir şekilde saklanıp işlendiği Veri Katmanı (Database) olmak üzere üç bağımsız birimden oluşmaktadır.

Frontend tarafında HTML5, CSS3 ve modern JavaScript (ES6+) kullanılarak oluşturulan istemci, REST tabanlı FastAPI backend sunucusuyla asenkron Fetch API üzerinden JSON formatında haberleşmektedir. Mantık katmanında yer alan Python/FastAPI uygulaması, Pydantic modelleriyle veri güvenliğini sağlarken, mysql-connector-python sürücüsü aracılığıyla MySQL veritabanına bağlanmaktadır. Veritabanı katmanı ise yalnızca bir veri deposu olarak değil; içerdiği tetikleyiciler (triggers) ve saklı yordamlar (stored procedures) sayesinde iş mantığının bir kısmını kendi üzerinde çalıştıran aktif bir mimari elemanı olarak görev yapmaktadır. Bu mimari izolasyon, projenin modülerliğini ve sürdürülebilirliğini maksimum düzeye çıkarmaktadır.

1.6. Projenin Klasör Yapısı

OTEL-OTOMASYONU/

```
|
|
|—— backend/                                # Mantık ve API Katmanı
|  |—— __pycache__/                          # Python derlenmiş önbellek dosyaları
|  |—— venv/                                # Python sanal ortam (virtual environment) klasörü
|  |—— .gitignore                           # Git versiyon kontrolü hariç tutma listesi
|  |—— .gitkeep                             # Boş klasör tutucu dosya
|  |—— database.py                          # MySQL Bağlantı ve Pool Yönetimi
|  |—— main.py                             # FastAPI Uygulama ve Endpoint Tanımları
|  |—— queries.py                           # Ham SQL Sorguları ve View Referansları
|  |—— requirements.txt                     # Proje bağımlılıkları (pip kütüphaneleri)
|
|—— database/                                # VTYS ve Şema Katmanı (SQL Scriptleri)
|  |—— 01_tablo_kurulumu.sql
|  |—— 02_ornek_veriler.sql
|  |—— 03a_tetikleyici_oda_durumu_guncelle.sql
|  |—— 03b_tetikleyici_tarih_kontrolu.sql
|  |—— 04a_sanalTablo_aktif_musteriler.sql
|  |—— 04b_sanalTablo_fatura_bekleyenler.sql
|  |—— 04c_sanalTablo_vw_dashboard_ozet.sql
|  |—— 04d_sanalTablo_vw_rezervasyon_listesi.sql
|  |—— 04e_sanalTablo_vw_oda_detaylari.sql
|  |—— 04f_sanalTablo_vw_aktif_borclar.sql
|  |—— 05a_sanalMethod_fatura_hesapla.sql
|  |—— 05b_sanalMethod_hizmet_ekleme.sql
|  |—— 05c_sanalMethod_yeni_rezervasyon_ekle.sql
|  |—— 05d_sanalMethod_musait_oda_Ara.sql
|  |—— 05e_sanalMethod_rezervasyon_durum_guncelle.sql
|  |—— 05f_sanalMethod_oda_turu_fiyat_guncelle.sql
|  |—— 05g_sanalMethod_hizmet_fiyat_guncelle.sql
|  |—— 99_tam_sifirlama.sql
|
|—— docs/                                    # Dokümantasyon Klasörü
|  |—— Veritabanı_Yönetim_Sistemleri_Dersi_Proje_Raporu.docx
```

	— frontend/	# Sunum Katmanı (Kullanıcı Arayüzü)
	— css/	
	— styles.css	# Ortak CSS Stil Dosyası
	— img/	# Kullanılan Fotoğrafların Olduğu Klasör
	— js/	# JavaScript Mantık ve API Dosyaları
	— api.js	
	— auth.js	
	— ayarlar.js	
	— common.js	
	— dashboard.js	
	— gecmis.js	
	— islemler.js	
	— misafirler.js	
	— odalar.js	
	— public.js	
	— rezervasyonlar.js	
	— yetkili-auth.js	
	— yetkili/	# Özel Yetkili Yönlendirme Klasörü
	— index.html	# Yetkili Giriş/Karşılama Sayfası
	— admin.html	
	— admin-ayarlar.html	
	— admin-islemler.html	
	— admin-misafirler.html	
	— admin-odalar.html	
	— admin-rezervasyonlar.html	
	— ayarlar.html	
	— dashboard.html	
	— frontend.html	
	— gecmis.html	
	— islemler.html	
	— misafirler.html	
	— odalar.html	

| | — rezervasyonlar.html

| | — yetkili.html

|

— README.md

Proje genel kurulum ve çalıştırma talimatları

1.7. Proje Modülleri

Sistemimiz, karmaşık otel operasyonlarını yönetebilmek için birbirleriyle entegre çalışan 6 ana modülden oluşmaktadır. Her bir modül, frontend arayüzünden başlayıp FastAPI backend mantığına ve MySQL veritabanındaki saklı yordam/tetikleyici yapılarına kadar uzanan tam yığın (full-stack) bir döngüyü kapsar.

1. Akıllı Rezervasyon ve Müşteri Yönetim Modülü

- **İşlevi:** Otel müşterilerinin sisteme kaydı, müsaitlik sorgulanması ve rezervasyon atamalarının yapıldığı ana operasyon modülüdür.
- **Teknik Altyapısı:** Veritabanında `sp_YeniRezervasyonEkle` saklı yordamı ile çalışır. T.C. kimlik numarası üzerinden müşterinin daha önce otelde kalıp kalmadığını kontrol eder. Yeni müşteriye otomatik kaydeder, ardından istenen tarihlerde çakışmayan ve "Arızalı" olmayan uygun odayı akıllı algoritmasıyla bularak atar. Ayrıca `trg_tarih_kontrol` tetikleyicisi sayesinde geçmiş tarihe veya hatalı aralıklara rezervasyon girilmesi veritabanı seviyesinde reddedilir.

2. Oda Durumu ve Kat Operasyonları Modülü

- **İşlevi:** Oteldeki tüm odaların (Boş, Dolu, Temizlikte, Arızalı) durumlarını kat planına göre gerçek zamanlı ve görsel olarak takip etmeyi sağlar.
- **Teknik Altyapısı:** Frontend tarafında `odalar.html` arayüzünde renkli bir matris olarak çalışır. Veritabanında `vw_oda_detaylari` sanal tablosundan beslenir. `trg_oda_durumu_guncelle` tetikleyicisi sayesinde, bir rezervasyon onaylandığında oda otomatik "Dolu", müşteri çıkış yaptığında ise "Temizlikte" durumuna geçer. Temizlik personeli odayı temizlediğinde tek tıkla oda tekrar "Boş" durumuna çekilir.

3. Finans, Fatura ve Check-out (Çıkış) Modülü

- **İşlevi:** Müşterinin konakladığı gün sayısı ve kaldığı süre boyunca aldığı tüm ekstra hizmetleri toplayarak nihai faturayı oluşturur ve tahsilat/çıkış işlemlerini kapatır.
- **Teknik Altyapısı:** `islemler.html` üzerinden yönetilir. Çıkış anında `sp_FaturaKes` prosedürü devreye girer. Bu prosedür `DATEDIFF` fonksiyonu ile giriş-çıkış tarihleri

arasındaki gün sayısını kullanarak oda ücretini hesaplar, ardından Rezervasyon_Hizmetleri tablosundaki kayıtları ekleyerek genel toplamı oluşturur ve faturayı keser. Faturası kesilmeyen ve güncel borç kaydı bulunan müşteriler vw_aktif_borclar sanal tablosu (view) üzerinden dinamik olarak sorgulanarak sistemde bekleyen ödemeler listesinde gösterilir.

4. Ekstra Hizmetler ve Dinamik Fiyatlandırma Modülü

- **İşlevi:** Minibar, SPA, VIP transfer gibi ek hizmetlerin misafirlerin hesaplarına anlık işlenmesini ve otel yönetiminin (Admin) oda/hizmet fiyatlarını dinamik olarak güncellemesini sağlar.
- **Teknik Altyapısı:** Misafire hizmet eklenirken sp_HizmetEkle prosedürü çalışır. Yönetici fiyat güncellemek istediğinde ise sp_OdaTuruFiyatGuncelle ve sp_HizmetFiyatGuncelle yordamları ile veritabanı tabloları anında güncellenir.

5. Dashboard (Pano) ve Sistem Loglama Modülü

- **İşlevi:** Otel yöneticisine o günkü aktif misafir sayısını, kasadaki toplam ciroyu ve beklenen çıkışları tek bir operasyon merkezinde sunar. Ayrıca sistemde yapılan tüm işlemleri denetim amacıyla kayıt altına alır.
- **Teknik Altyapısı:** Sistemde oda durumu değişmesi, rezervasyon onaylanması veya fatura kesilmesi gibi tüm kritik işlemler, istemci tarafında yüksek performanslı ve anlık takibe izin veren localStorage loglama yapısıyla asenkron olarak yönetilmekte ve gecmis.html üzerinden kronolojik olarak izlenebilmektedir.

6. Kimlik Doğrulama (Auth) ve Yetkilendirme Modülü

- **İşlevi:** Müşteri paneli ile personel yönetim panelini ayırarak, sisteme sadece yetkili personelin (Admin, Resepsiyonist vb.) güvenli bir şekilde erişmesini sağlar.
- **Teknik Altyapısı:** Personeller tablosundaki kimlik bilgileri, FastAPI backend'indeki /login uç noktasından doğrulanır. Başarılı bir kimlik doğrulaması sırasında istemci tarafında session kontrolü amacıyla localStorage üzerinde bir oturum token'i oluşturulur. Yetkisiz erişimler tarayıcı tabanlı yönlendirme mantığı ile kontrol edilerek yönetim paneli güvenliği simüle edilir.

2. PROJE YÖNETİMİ

2.1.Takım Üyeleri Tanımı

Üye No	Ad-Soyad	Öğrenci No	Görevi (Takım Lideri/Takım Üyesi)
1	Uğur Erdoğan	240290020	Takım Lideri / Backend Geliştirici & API Tasarımcısı
2	Hilal Çoğul	240290090	Takım Üyesi / Veritabanı Yöneticisi (DBA) & SQL Mimarı
3	Ecenur Eke	250290096	Takım Üyesi / UI/UX Tasarımcısı & Frontend Geliştirici

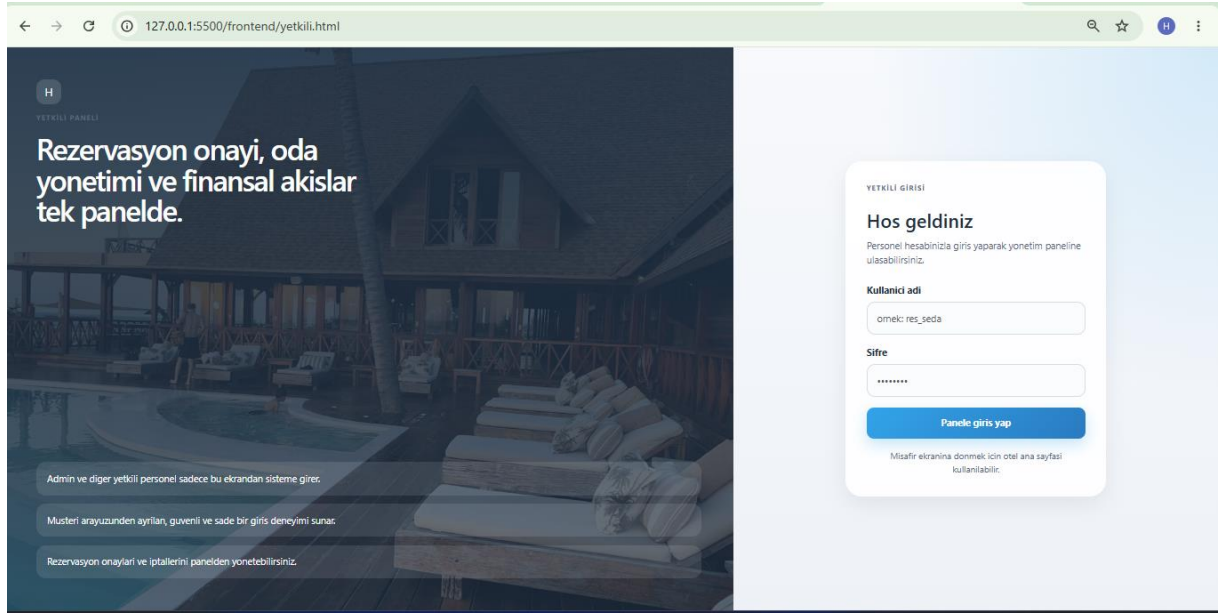
2.2.Görev Dağılımı

İş Paketi No	İş Paketinin Adı ve Hedefleri	Görevlendirilen Takım Üyesi
WP1	Veritabanı Şema Tasarımı ve Veri Bütünlüğü (DDL): Rezervasyon, Müşteri, Finans ve Oda Yönetimi modüllerinin altyapısını oluşturan ilişkisel tabloların (1NF, 2NF, 3NF standartlarında) tasarlanması ve PK/FK/Unique kısıtlamalarının sisteme entegrasyonu.	Hilal Çoğul
WP2	Gelişmiş VTYS Programlama ve İş Kuralları (Programmable SQL): Akıllı Rezervasyon Modülü (sp_YeniRezervasyonEkle), Finans Modülü (sp_FaturaKes) ve Oda Operasyonları Modülü (trg_oda_durumu_guncelle) için gerekli saklı yordam, tetikleyici ve sanal tabloların (view) kodlanması.	Hilal Çoğul
WP3	Backend REST API Altyapısı ve Güvenlik: FastAPI web framework'ü	Uğur Erdoğan

	kullanılarak projenin sunucu iskeletinin kurulması, MySQL bağlantı havuzunun oluşturulması, Kimlik Doğrulama Modülü ve veri doğrulama (Pydantic) modellerinin yapılandırılması.	
WP4	API İş Mantığı (Endpoint) Entegrasyonu: Pano (Dashboard), Ekstra Hizmetler ve Rezervasyon modüllerinin veritabanı yordamlarıyla haberleşmesini sağlayan asenkron API uç noktalarının (endpoints) geliştirilmesi ve HTTP hata yönetiminin (exception handling) sağlanması.	Uğur Erdoğan
WP5	Arayüz Tasarımı (UI/UX) ve Görsel İskelet: Operasyon Panosu (Dashboard), Oda Durum Takip, Rezervasyon ve Fatura ekranlarının Bootstrap v5 kütüphanesi ile modern, mobil uyumlu (responsive) ve kullanıcı dostu bir yapıda (HTML/CSS) tasarlanması.	Ecenur Eke
WP6	Asenkron İstemci (Frontend) Geliştirme ve API Haberleşmesi: Arayüz tasarımlarının fetch API ile backend uç noktalarına bağlanması; rezervasyon oluşturma, fatura kesme, durum güncelleme gibi modül işlemlerinin JavaScript (DOM manipülasyonu) ile dinamik hale getirilmesi.	Ecenur Eke

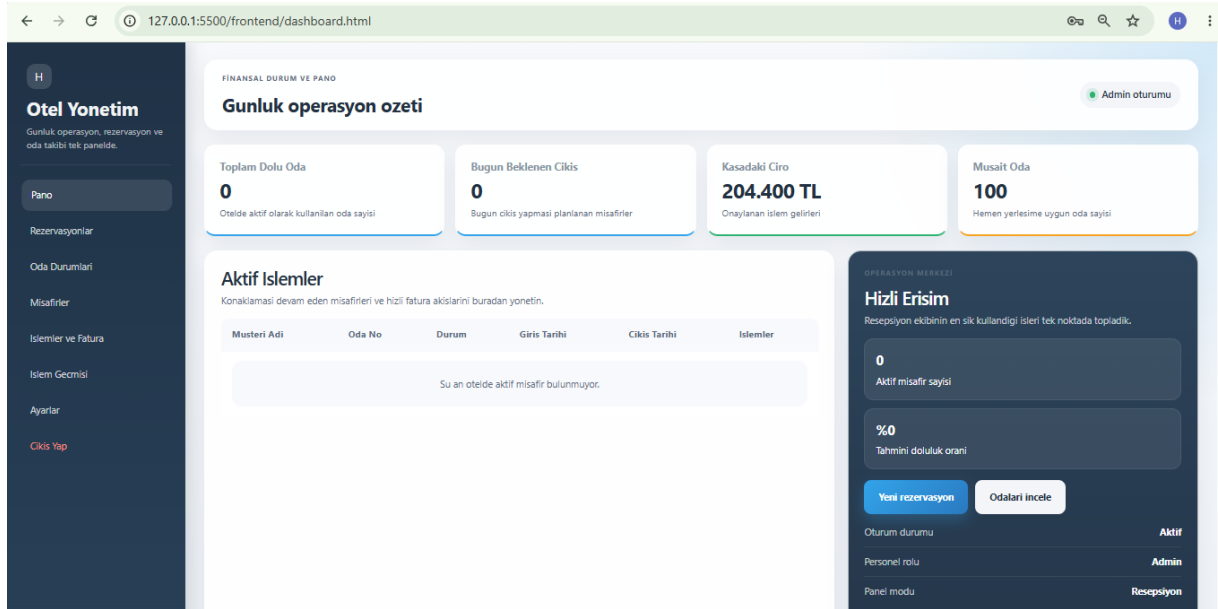
3. SONUÇLAR

3.1.Ekran Çıktıları



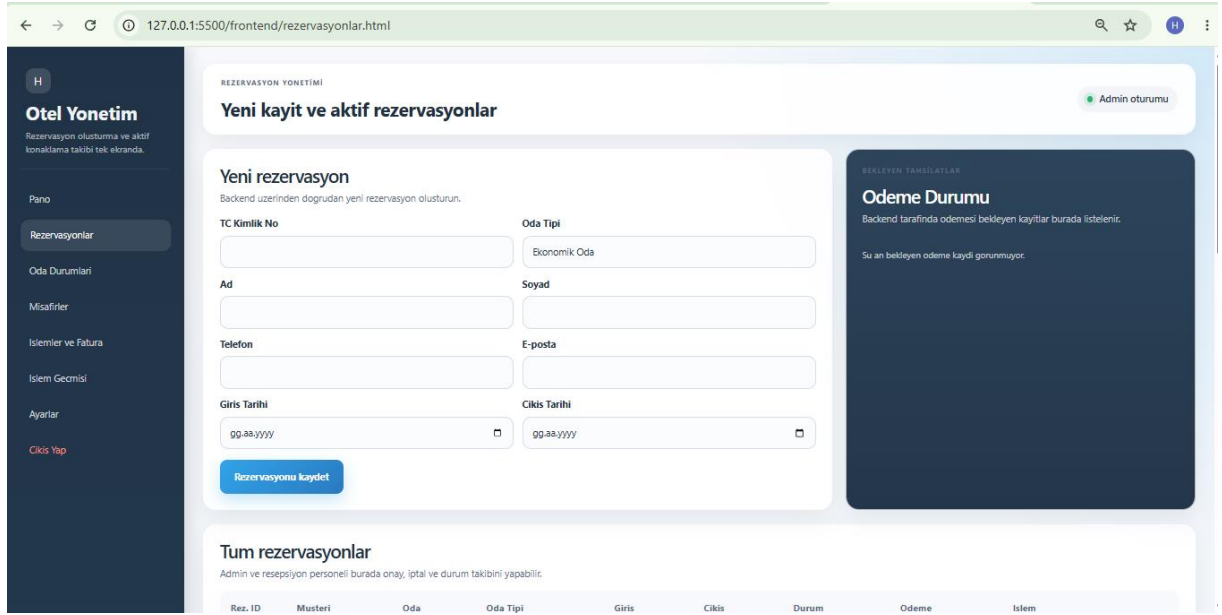
Şekil 3.1: Yetkili Girişi (Login) Ekranı

İlgili Bileşen: Kimlik Doğrulama (Auth) ve Yetkilendirme Modülü. Personel kimlik bilgilerinin backend katmanında doğrulanarak istemci tarafında session tabanlı geçici token (localStorage) kontrolünün simüle edildiği ve yetkisiz panel erişimlerinin tarayıcı tabanlı yönlendirme mantığıyla engellendiği personel giriş arayüzüdür.



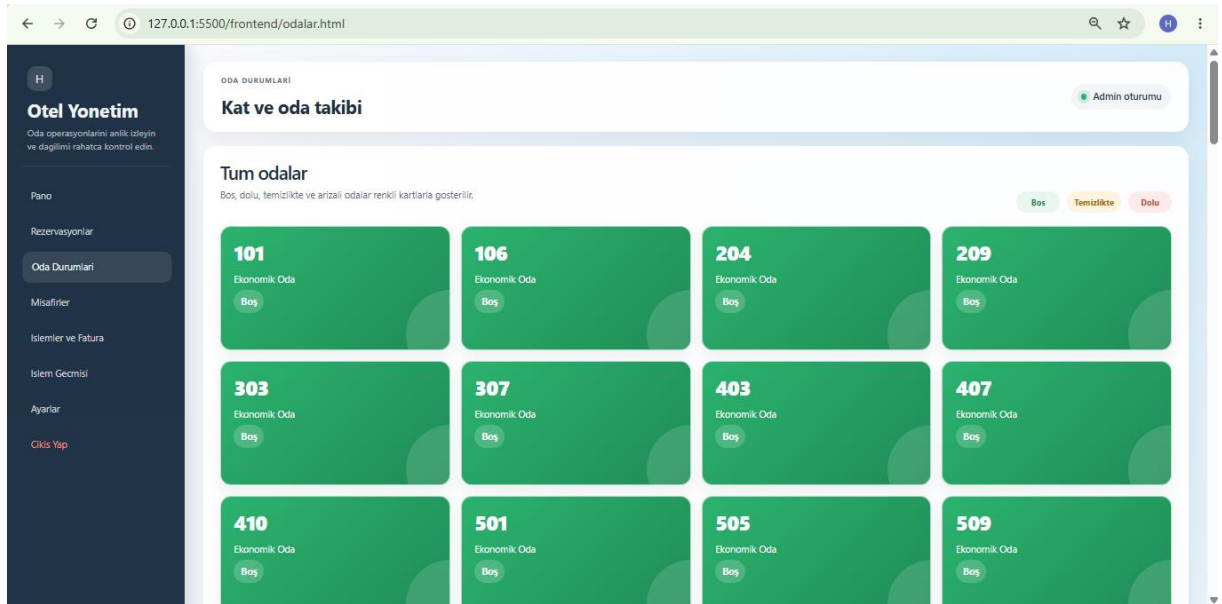
Şekil 3.2: Yönetim Paneli (Dashboard) ve Operasyon Özeti

İlgili Bileşen: Dashboard (Pano) Modülü. vw_dashboard_ozet sanal tablosundan beslenerek otelin anlık doluluk oranını, aktif misafir sayısını ve kasadaki toplam ciroyu tek ekranda sunan kontrol merkezidir.



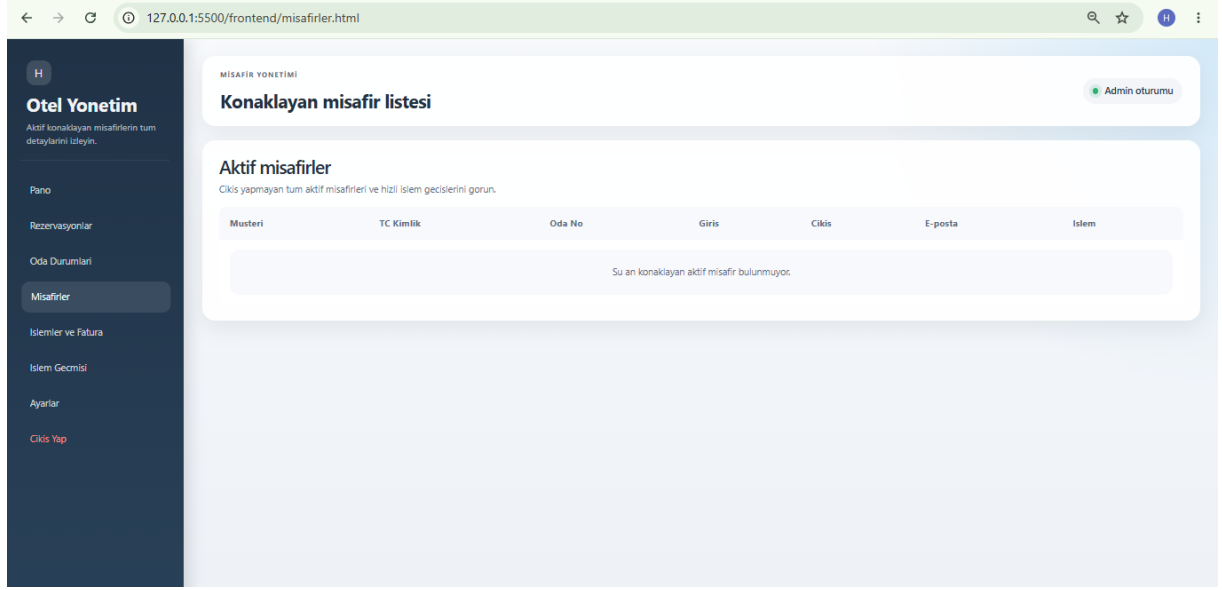
Şekil 3.3: İç Sistem Yeni Rezervasyon ve Liste Ekranı

İlgili Bileşen: Akıllı Rezervasyon ve Müşteri Yönetim Modülü. Resepsiyon personelinin `sp_YeniRezervasyonEkle` prosedürünü kullanarak çakışmasız oda ataması yaptığı ve rezervasyonları onayladığı ekrandır.



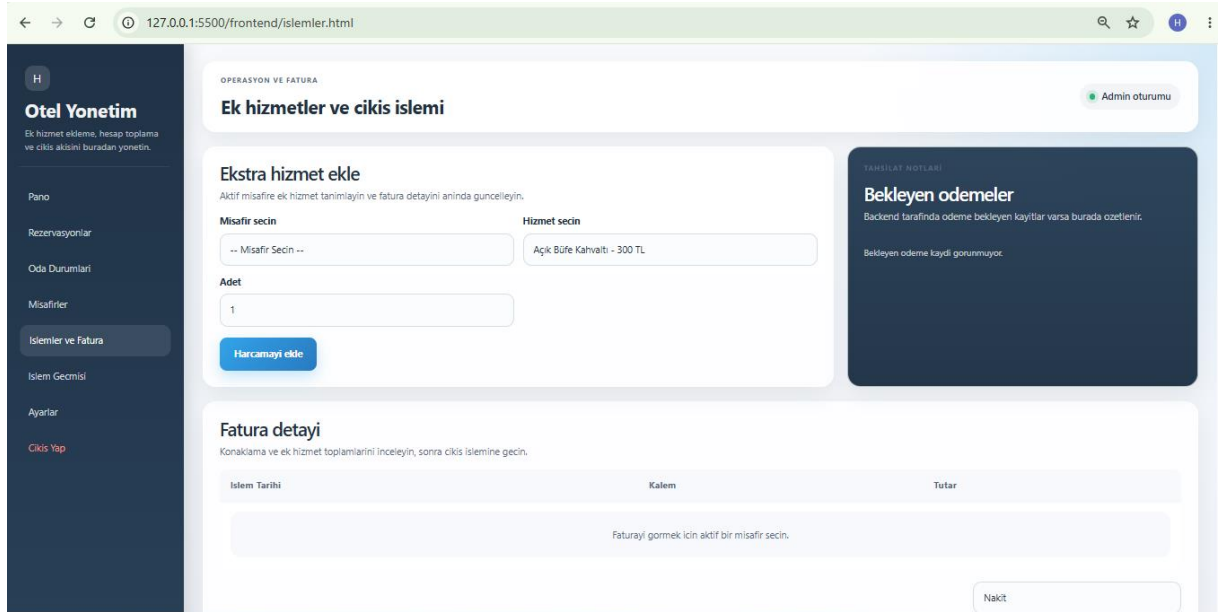
Şekil 3.4: Dinamik Oda Durumları (Kat Planı) Ekranı

İlgili Bileşen: Oda Durumu ve Kat Operasyonları Modülü. Oteldeki tüm odaların anlık durumlarını renkli matris mimarisiyle gösteren ve temizlik işlemlerinin yönetildiği arayüzdür.



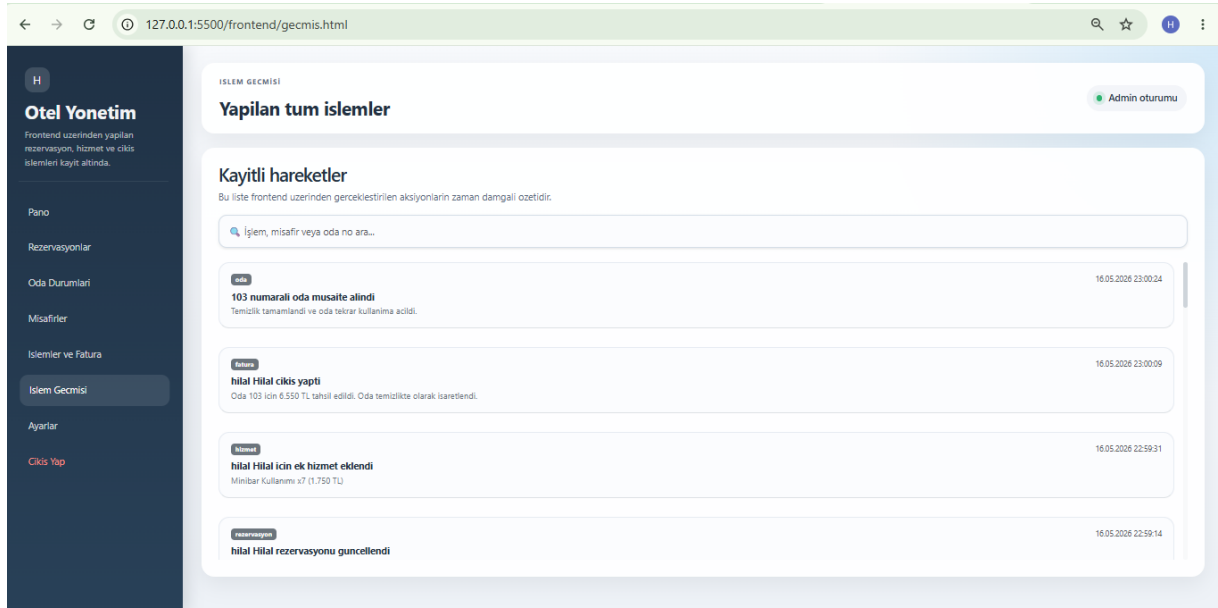
Şekil 3.5: Aktif Konaklayan Misafirler Ekranı

İlgili Bileşen: Müşteri Yönetim Modülü. vw_aktif_musteriler sanal tablosu aracılığıyla o an otelde konaklayan müşterilerin bilgilerinin listelendiği yönetim arayüzüdür.



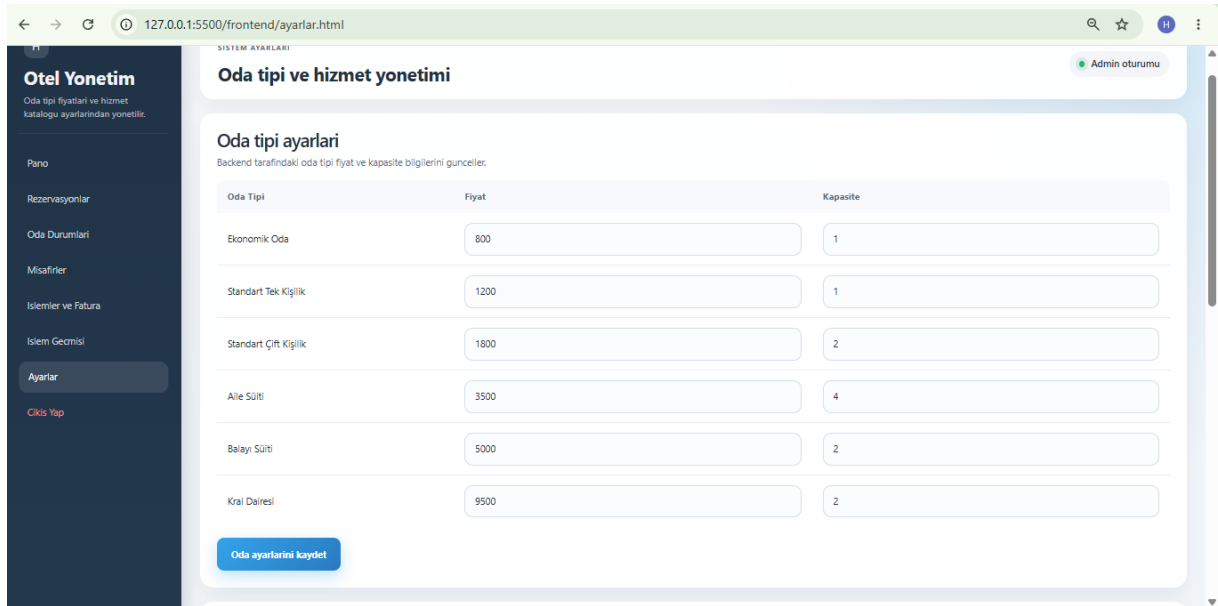
Şekil 3.6: İşlemler, Ekstra Hizmetler ve Fatura Ekranı

İlgili Bileşen: Finans, Fatura ve Check-out Modülü. Konaklayan misafirlere ekstra hizmetlerin eklendiği ve çıkış anında sp_FaturaKes prosedürü tetiklenerek nihai hesabın kapatıldığı ödeme ekranıdır.



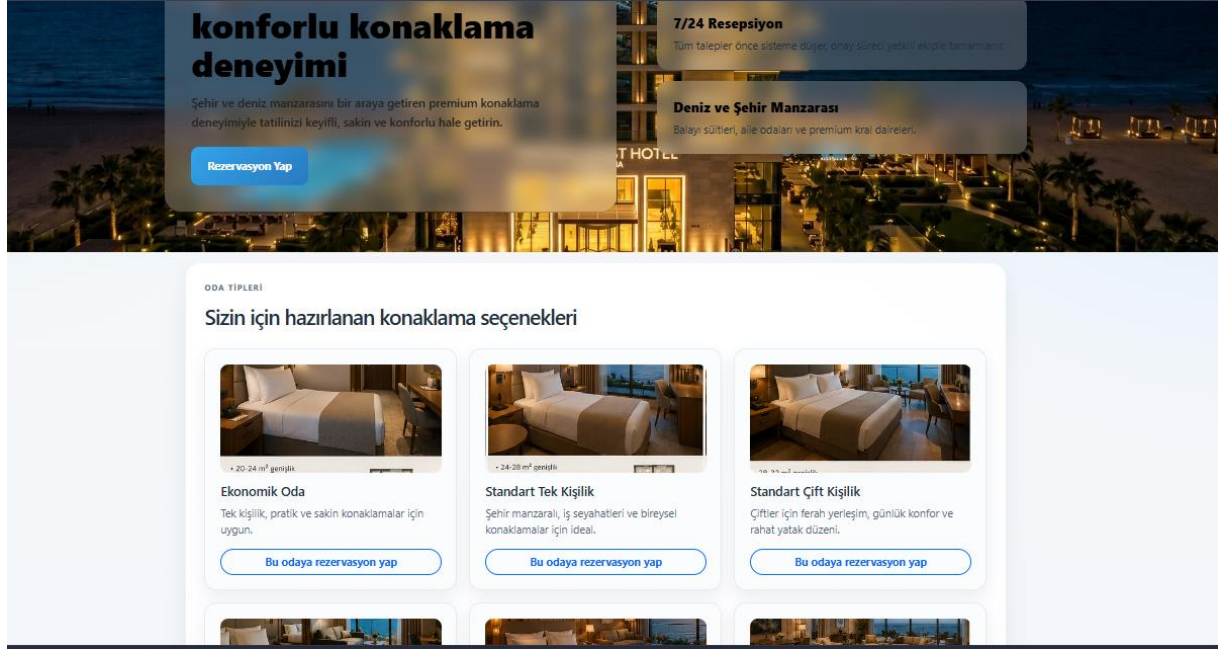
Şekil 3.7: Sistem İşlem Geçmişi (Log) Ekranı

İlgili Bileşen: Sistem Loglama Modülü. Arayüz üzerinden gerçekleştirilen tüm operasyonların ve durum güncellemelerinin, istemci tarafında (localStorage) anlık kronolojik log yapısıyla kayıt altına alınarak asenkron olarak listelendiği denetim ekranıdır.



Şekil 3.8: Dinamik Fiyatlandırma ve Sistem Ayarları Ekranı

İlgili Bileşen: Ekstra Hizmetler ve Dinamik Fiyatlandırma Modülü. Yönetici (Admin) yetkisiyle oteldeki oda türlerinin taban fiyatlarının ve ekstra hizmet ücretlerinin (SPA, Minibar vb.), veritabanındaki sp_OdaTuruFiyatGuncelle ve sp_HizmetFiyatGuncelle saklı yordamları tetiklenerek anında güncellendiği yönetim arayüzüdür.



Şekil 3.9: Müşteri Karşılama (Landing Page) ve Oda Tanıtım Ekranı

İlgili Bileşen: Otel müşterilerinin sisteme ilk erişim sağladığı; otel konseptinin, oda türlerinin ve kapasite standartlarının modern ve anlamsal HTML5/CSS3 mimarisiyle statik olarak sergilendiği ana vitrin arayüzüdür.

3.2. Program Kodları

Projenin kodlarının bulunduğu Drive linki:

https://drive.google.com/drive/folders/1x19OqUvVCqtMFPkJ4YexgtCznZencJVH?usp=drive_link

Bu bölümde, otel otomasyon sisteminin mantık katmanını (Logic Layer) oluşturan FastAPI asenkron uç noktalarından (endpoints) kritik olanları, ilişkili veritabanı yordamları ve veri akış mimarisiyle birlikte sunulmuştur.

A. Gösterge Paneli ve Merkezi Operasyon Özeti API Entegrasyonu

Kontrol paneli (Dashboard), ağ trafiğini ve veritabanı üzerindeki sorgu yükünü minimize etmek amacıyla tek bir uç nokta üzerinden çoklu veri kümesi dönmektedir. Sistem, anlık doluluk durumunu ve boş oda sayılarını dinamik olarak hesaplarken, ciro bilgilerini ilgili View yapısı üzerinden asenkron olarak birleştirir. Ayrıca CURDATE() fonksiyonuyla entegre çalışan bir alt sorgu sayesinde bugün çıkış yapması beklenen misafir sayısını anlık takip eder.

```
@app.get("/dashboard/ozet")
def dashboard_verilerini_getir():
    conn = get_db_connection()
    if not conn:
```

```

        return {"dolu_oda_sayisi": 0, "bos_oda_sayisi": 0, "toplaml_ciro": 0,
"bugun_cikis_yapacaklar": 0}

    try:
        cursor = conn.cursor(dictionary=True)

        # 1. Dolu ve Boş Oda Sayısını Bulma
        cursor.execute(queries.GET_AKTIF_MUSTERILER)
        aktif_misafirler = cursor.fetchall()
        gercek_dolu_oda = len(aktif_misafirler)

        cursor.execute("SELECT COUNT(*) as toplam FROM Odalar")
        toplam_oda_sonuc = cursor.fetchone()
        toplam_oda = toplam_oda_sonuc["toplam"] if toplam_oda_sonuc else 0
        gercek_bos_oda = toplam_oda - gercek_dolu_oda

        # 2. Ciro ve Özet Bilgisi
        cursor.execute(queries.GET_DASHBOARD_OZET)
        ozet = cursor.fetchone()

        # Bugün Çıkış Yapacak Misafir Sayısı
        # (Otomatik olarak bugünün tarihini kontrol eder)
        bugun_sorgusu = """
            SELECT COUNT(*) as cikis_sayisi
            FROM Rezervasyonlar
            WHERE rezerve_durumu NOT IN ('İptal Edildi', 'Tamamlandı')
            AND rezerve_cikis_tarihi = CURDATE()
        """
        cursor.execute(bugun_sorgusu)
        bugun_cikis_sonuc = cursor.fetchone()
        bugun_cikis = bugun_cikis_sonuc["cikis_sayisi"] if bugun_cikis_sonuc else 0

        # 4. Frontend'e gidecek standart paket
        sonuc = {
            "dolu_oda_sayisi": gercek_dolu_oda,
            "bos_oda_sayisi": gercek_bos_oda,
            "bugun_cikis_yapacaklar": bugun_cikis,
            "toplaml_ciro": 0.0
        }

        # SQL'den dönen ciro kolonunun adı ne olursa olsun yakalıyoruz
        if ozet:
            ciro_degeri = ozet.get("toplaml_ciro") or ozet.get("ciro") or ozet.get("toplaml_tutar") or 0
            sonuc["toplaml_ciro"] = float(ciro_degeri)

        return sonuc

    except Exception as e:

```

```

print("Dashboard Hatası:", str(e))
return {"dolu_oda_sayisi": 0, "bos_oda_sayisi": 0, "toplam_ciro": 0,
"bugun_cikis_yapacaklar": 0}
finally:
    if conn and conn.is_connected():
        cursor.close()
        conn.close()

```

B. Dinamik Oda Türü Yapılandırması ve Fiyat-Kapasite Senkronizasyonu

Yönetici panelinden tetiklenen oda türü güncellenmesi, veritabanı katmanındaki ilişkisel şemanın korunması adına doğrudan sp_OdaTuruGuncelle saklı yordamı (Stored Procedure) çağrılarak yürütülür. Bu uç nokta; oda türü adını, yeni fiyat politikasını ve güncellenen maksimum kapasite kriterlerini tek bir işlem kümesiyle veritabanına parametrik olarak iletir.

```

@app.put("/ayarlar/oda-fiyat")
def oda_fiyati_guncelle(veri: FiyatGuncelleme):
    conn = get_db_connection()
    try:
        cursor = conn.cursor()
        cursor.callproc("sp_OdaTuruGuncelle", (veri.oda_turu_adi, veri.yeni_fiyat,
veri.yeni_kapasite))
        conn.commit()
        return {"mesaj": "Oda turu fiyatı başarıyla guncellendi."}
    finally:
        conn.close()

```

C. Finans Katmanı ve Aktif Borç Takip Mekanizması

Sistemde faturalandırılmayı bekleyen veya ara harcama dökümleri incelenmek istenen misafirlerin finansal kayıtları, veritabanı seviyesinde optimize edilmiş olan vw_aktif_borclar sanal tablosu (View) üzerinden çekilir. Bu asenkron metot, fatura kesim aşaması öncesinde veritabanı verilerinin güncel halini arayüze aktarır.

```

@app.get("/finans/odeme-bekleyenler")
def odeme_bekleyenleri_getir():
    conn = get_db_connection()
    if not conn:
        return []

    try:
        cursor = conn.cursor(dictionary=True)
        # Artık veritabanındaki hazır View'ı çağırıyoruz
        cursor.execute(queries.GET_AKTIF_BORCLAR)
        return cursor.fetchall()
    except Exception as e:

```



```

        print("Bekleyen Ödeme Hatası:", str(e))
    return []
finally:
    if conn and conn.is_connected():
        cursor.close()
        conn.close()

```

D. Personel Oturum Yönetimi ve Kimlik Doğrulama Altyapısı

Sisteme erişmek isteyen personelin yetkileri ve rolleri, veritabanındaki Personeller tablosundaki kayıtlar üzerinden filtrelendir. Başarılı giriş işlemlerinde backend katmanı, tarayıcı tabanlı yönlendirme kurallarını ve oturum sürekliliğini istemci katmanında (localStorage) güvenli bir şekilde simüle etmek amacıyla benzersiz bir session token mimarisi döndürür.

```

@app.post("/login")
def sisteme_giris_yap(bilgiler: PersonelGiris):
    conn = get_db_connection()
    if not conn:
        raise HTTPException(status_code=500, detail="Veritabani baglantisi kurulamadi")

    try:
        cursor = conn.cursor(dictionary=True)
        cursor.execute(queries.GET_PERSONEL_GIRIS, (bilgiler.kullanici_adi, bilgiler.sifre))
        personel = cursor.fetchone()
        if personel:
            return {
                "mesaj": "Giris basarili",
                "token": f"fake-jwt-token-{personel['personel_id']}",
                "role": personel["personel_rol"],
            }

        raise HTTPException(status_code=401, detail="Gecersiz kullanıcı adı veya şifre")
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
    finally:
        if conn and conn.is_connected():
            cursor.close()
            conn.close()

```

E. Rezervasyon Bazlı Ekstra Hizmet Tüketimlerinin İlişkisel Listelenmesi

Konaklayan misafirlerin kaldıkları süre boyunca yararlandıkları ekstra harcamalar (SPA, Minibar vb.), rezervasyon_hizmetleri ara tablosu ile hizmetler ana tablosunun dinamik bir şekilde JOIN edilmesiyle listelenir. Bu uç nokta, her bir kalemin adedini ve birim fiyatını anlık ilişkisel modelden çekerek doğru hesap dökümünün oluşturulmasını sağlar.

```

@app.get("/rezervasyonlar/{rezervasyon_id}/hizmetler")

```

```

def rezervasyon_hizmetlerini_getir(rezervasyon_id: int):
    conn = get_db_connection()
    try:
        cursor = conn.cursor(dictionary=True)
        sorgu = """
            SELECT rh.hizmet_id, h.hizmet_adi, h.hizmet_birim_fiyat, rh.hizmet_adet
            FROM rezervasyon_hizmetleri rh
            JOIN hizmetler h ON rh.hizmet_id = h.hizmet_id
            WHERE rh.rezervasyon_id = %s
        """
        cursor.execute(sorgu, (rezervasyon_id,))
        return cursor.fetchall()
    finally:
        conn.close()

```

F. Veritabanı Bağlantı Yönetimi (database.py)

Sistemin MySQL veritabanı ile kesintisiz iletişim kurmasını sağlayan temel bağlantı modülüdür. İşletim sistemi seviyesindeki olası IPv6 (localhost) DNS çözümleme çakışmalarını önlemek ve ağ gecikmesini düşürmek adına, bağlantı doğrudan IPv4 (127.0.0.1) protokolü üzerinden sağlanmıştır. Ayrıca bağlantı konfigürasyonuna eklenen use_pure=True parametresi sayesinde, veritabanı sürücüsünün platform bağımsız (pure Python) çalışması zorlanarak olası C eklentisi (C-extension) derleme hatalarının önüne geçilmiştir. Veritabanı erişiminde yaşanabilecek olası kesintiler, hata yakalama (Try-Except) blokları ile kontrol altına alınmıştır.

```

import mysql.connector
from mysql.connector import Error

def get_db_connection():
    try:
        connection = mysql.connector.connect(
            host='127.0.0.1',
            database='otel_otomasyonu',
            user='root',
            password='',
            use_pure=True
        )
        if connection.is_connected():
            return connection
    except Error as e:
        print(f"Veritabanı bağlantı hatası: {e}")
        return None

```

3.3. Sorgular

Bu bölüm, projenin temel SQL programlama yapılarını içerir.

- 01_tablo_kurulumu.sql : Veritabanı tablolarının kurulması için gereken sql sorgularını barındırır.
- 02_ornek_veriler.sql : Veritabanı tablolarında olması gereken test verilerini içerir.
- 99_tam_sifirlama.sql : Veritabanı tablolarının tamamen sıfırlanmasını sağlayan sql sorgularını içerir.

3.3.1. Tetikleyiciler (Triggers)

- 03a_tetikleyici_oda_durumu_guncelle.sql : Rezervasyon durumu değiştiğinde oda durumunun otomatik güncellenmesini sağlayan sorguyu içerir.
- 03b_tetikleyici_tarih_kontrolu.sql : Rezervasyon tarih kontrolünü sağlayan sql sorgusunu içerir. Rezervasyon alımı sırasında kullanılır.

3.3.2. Sanal Tablolar (Views)

Uygulama içinde sıkça join yapılan karmaşık sorguları tek bir sanal tabloda toplayarak performans kazanımı sağlanmıştır.

- 04a_sanalTablo_aktif_musteriler.sql : Otelde mevcut gün içerisinde bulunan müşterilerin listelenmesini sağlayan sorguyu içerir. Bu sorgu dashboard.html sayfasında aktif işlemler tablosunu doldurmak için kullanılmıştır.
- 04b_sanalTablo_fatura_bekleyenler.sql : Henüz faturası kesilmemiş müşterilerin listelenmesini sağlayan sql sorgusunu içerir. Bu sorgu işlemler.html dosyasındaki bekleyen ödemeler kısmının doldurulması için kullanılmıştır.
- 04c_sanalTablo_vw_dashboard_ozet.sql : Admin panelinde genel istatistiklerin bulunmasını sağlayan sql sorgusunu içerir. Bu sorgu dashboard.html sayfasında bulunan günlük operasyon özeti kartlarındaki verileri doldurmak için kullanılmıştır.
- 04d_sanalTablo_vw_rezervasyon_listesi.sql : Oteldeki mevcut rezervasyonların listelenmesini sağlayan sql sorgusunu içerir. Bu sorgu admin kısmındaki rezervasyonlar sayfasının içeriğini oluşturmaktadır.
- 04e_sanalTablo_vw_oda_detaylari.sql : Odaların genel bilgilerinin listelenmesini sağlayan sql sorgusunu içerir. Bu sorgu admin kısmındaki oda durumlari sayfasının içeriğini oluşturmaktadır.
- 04f_sanalTablo_vw_aktif_borclar.sql : Hizmet alımları ve konaklama süreleri sonrası henüz çıkışı yapılmamış ve güncel borç kaydı bulunan müşterilerin finansal durumunu

listeler. islemler.html sayfasındaki bekleyen ödemeler tablosu bu görünümünden beslenmektedir.

3.3.3. Saklı Yordamlar (Stored Procedures)

- 05a_sanalMethod_fatura_hesapla.sql : Müşteriye ait rezervasyonun faturasının kesilmesi işlemi için gereken işlemleri sağlayan sql sorgusunu içerir. İşlemler ve faturalar sayfasında kullanılmıştır.
- 05b_sanalMethod_hizmet_ekleme.sql: Konaklayan müşterilerin (rezervasyonların) aldıkları ekstra otel hizmetlerini (SPA, Minibar vb.) adet bilgisiyle birlikte Rezervasyon_Hizmetleri tablosuna güvenli bir şekilde işleyen saklı yordam sorgusunu içerir.
- 05c_sanalMethod_yeni_rezervasyon_ekle.sql : Yeni rezervasyon eklenmesini sağlayan sql sorgusunu içerir. Rezervasyon ekleme kısmında kullanılmıştır.
- 05d_sanalMethod_musait_oda_Ara.sql : Yeni rezervasyon yapılacağında boşta bulunan odaların listelenmesini sağlayan sql sorgusunu içerir. Müşteri arayüzünde rezervasyon oluşturma aşamasında kullanılmıştır.
- 05e_sanalMethod_rezervasyon_durum_guncelle.sql : Rezervasyonun durumunun güncellenmesini sağlayan sql sorgusunu içerir. Admin kısmında rezervasyonlar sayfasında kullanılmıştır.
- 05f_sanalMethod_oda_turu_fiyat_guncelle.sql : Oda türlerinin fiyatlarının güncellenmesini sağlayan sql sorgusunu içerir. Admin kısmında ayarlar sayfasında kullanılmıştır.
- 05g_sanalMethod_hizmet_fiyat_guncelle.sql : Oteldeki hizmetlerin admin panelinde güncellenmesini sağlayan sql sorgusunu içerir. . Admin kısmında ayarlar sayfasında kullanılmıştır.